

LAPORAN ANALISIS TEKNIS PROYEK

KibokGO UNIB — AI-Powered Campus Navigation

Nama Proyek	KibokGO UNIB — AI-Powered Campus Navigation
Nama	Muhammad Iqbal Zafarullah
NIM	G1A024007
Dosen Pengampu	Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP
Mata Kuliah	Kecerdasan Buatan (Artificial Intelligence) - UAS
Institusi	Universitas Bengkulu (UNIB)
Tanggal Analisis	04 Juni 2026
NIM	G1A024007

1. PENDAHULUAN

1.1 Latar Belakang

Universitas Bengkulu (UNIB) memiliki kawasan kampus yang cukup luas yang terdiri dari berbagai dekanat fakultas, gedung perkuliahan bersama (GB), laboratorium, perpustakaan, sarana olahraga, kantin, area parkir, serta gerbang masuk utama. Kompleksitas tata letak gedung dan jalan di dalam lingkungan kampus sering kali membingungkan mahasiswa baru, tamu, maupun sivitas akademika lainnya dalam menentukan rute optimal atau lintasan terpendek dari satu titik ke titik lainnya.

Dalam bidang Ilmu Komputer dan Kecerdasan Buatan, pencarian lintasan terpendek (Shortest Path Problem) merupakan masalah klasik dalam teori graf yang memiliki aplikasi praktis tinggi. Algoritma Dijkstra adalah salah satu metode yang paling handal untuk menyelesaikan masalah ini pada graf berbobot positif. Proyek 'KibokGO UNIB' ini dirancang untuk menerapkan Algoritma Dijkstra guna memodelkan jaringan jalan kampus dan menyajikan solusi pencarian rute terpendek berbasis web secara interaktif dan real-time.

Aplikasi hasil implementasi proyek ini dapat diakses secara publik melalui tautan berikut:

<https://miqbalzafarullah.github.io/KibokGO-UNIB-ShortestPath/>

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam proyek ini adalah:

1. Bagaimana memodelkan tata letak gedung dan jaringan jalan di Universitas Bengkulu menjadi bentuk representasi graf (simpul dan sisi)?
2. Bagaimana mengimplementasikan Algoritma Dijkstra pada client-side web browser untuk mencari jalur terpendek?
3. Bagaimana mengintegrasikan hasil perhitungan algoritma dengan peta interaktif menggunakan OpenStreetMap (OSRM) dan Leaflet.js?
4. Bagaimana menyajikan visualisasi proses perhitungan agar mudah dipahami secara edukatif oleh pengguna?

1.3 Tujuan Proyek

Tujuan dari pelaksanaan proyek ini adalah:

1. Membuat visualisasi peta digital kampus Universitas Bengkulu berbasis web.
2. Mengimplementasikan Algoritma Dijkstra dalam JavaScript untuk pencarian lintasan terpendek antar gedung.
3. Menyediakan estimasi jarak tempuh dan waktu perjalanan (ETA) berdasarkan variasi moda transportasi (Jalan Kaki, Motor, dan Mobil).
4. Menyajikan log komputasi langkah demi langkah algoritma Dijkstra sebagai media pembelajaran mata kuliah Kecerdasan Buatan.

2. ANALISIS SISTEM

2.1 Deskripsi Sistem

KibokGO UNIB merupakan aplikasi web navigasi berbasis SPA (Single Page Application). Aplikasi ini berjalan sepenuhnya di browser pengguna (client-side) tanpa membutuhkan backend database yang rumit, sehingga memiliki performa pemuatan yang cepat dan efisiensi tinggi. Aplikasi ini menampilkan peta interaktif Universitas Bengkulu, daftar input pencarian berbasis autocomplete, penunjuk moda transportasi, panel detail rute, serta log perhitungan algoritma secara rinci.

2.2 Arsitektur Sistem

Sistem ini bekerja dengan skema integrasi dua lapis (Hybrid Routing Engine):

Lapis 1: Algoritma Dijkstra Lokal. Sistem membaca koordinat dan adjacency list 30 simpul utama yang merepresentasikan gedung dan persimpangan di UNIB dari file `locations.js`.

Lapis 2: Perhitungan Dijkstra. Ketika tombol 'Periksa Rute' ditekan, algoritma Dijkstra pada file `dijkstra.js` akan mencari jalur terpendek logis (urutan simpul/gedung yang harus dilalui).

Lapis 3: Routing Peta Nyata. Urutan koordinat simpul hasil Dijkstra dikirimkan ke OSRM (Open Source Routing Machine) API melalui request HTTPS untuk mendapatkan jalur geometris jalan yang nyata di peta.

Lapis 4: Rendering Visual. Leaflet.js merender polyline rute jalan di atas peta bersamaan dengan instruksi langkah-demi-langkah (turn-by-turn navigation) di panel samping.

3. ANALISIS ALGORITMA DIJKSTRA

3.1 Pemodelan Graf

Representasi graf jaringan jalan Universitas Bengkulu terdiri dari:

1. Simpul (Vertex/Node): Merepresentasikan gedung fakultas, gedung bersama, masjid, kantin, gerbang, dan persimpangan utama. Sistem mendefinisikan 30 simpul lokasi aktif.
2. Sisi (Edge): Jaringan jalan penghubung antar simpul.

3. Bobot (Weight): Jarak fisik antar gedung dalam satuan meter (m) yang diukur secara akurat berdasarkan jarak sebenarnya.

3.2 Pseudocode Algoritma

Implementasi Dijkstra pada berkas dijkstra.js adalah sebagai berikut:

```
function runDijkstra(startId, endId):  
    distances = {all_nodes: Infinity}  
    predecessors = {all_nodes: null}  
    distances[startId] = 0  
    queue = [{id: startId, dist: 0}]  
    visited = Set()  
  
    while queue is not empty:  
        sort queue by dist ascending  
        current = queue.shift()  
        u = current.id  
  
        if u in visited: continue  
        visited.add(u)  
  
        if u == endId: break  
  
        for neighbor v of u:  
            alt = distances[u] + weight(u, v)  
            if alt < distances[v]:  
                distances[v] = alt  
                predecessors[v] = u  
                queue.push({id: v, dist: alt})  
  
    return reconstruct_path(predecessors, endId)
```

3.3 Penjelasan Log Relaksasi

Proses relaksasi sisi (edge relaxation) adalah inti dari algoritma ini. Bila sistem menemukan bahwa jarak tersimpan ke simpul tujuan sementara ($\text{distances}[v]$) lebih besar daripada jarak ke simpul saat ini ditambah bobot sisi penghubung ($\text{distances}[u] + \text{weight}(u, v)$), maka nilai $\text{distances}[v]$ akan diperbarui dengan nilai baru yang lebih kecil. Log relaksasi ini dicatat secara detail pada UI dalam bentuk format HTML guna memberikan wawasan akademis tentang jalannya iterasi algoritma secara langsung kepada pengguna.

4. ANALISIS STRUKTUR KODE DAN MODUL

4.1 Struktur Data Lokasi (locations.js)

Modul `locations.js` menampung objek `nodes` yang berisi daftar koordinat (lat, lng), nama deskriptif lokasi, tipe (building, facility, parking), dan deskripsi lengkap. Selain itu, modul ini berisi logika autocomplete pencarian menggunakan penanganan event focus, input, dan klik yang didukung riwayat pencarian (search history) di penyimpanan lokal (localStorage).

4.2 Logika Algoritma (dijkstra.js)

Modul `dijkstra.js` menyimpan objek `DIJKSTRA_GRAPH` yang direpresentasikan sebagai Adjacency List. Setiap kunci (key) berisi objek tetangganya beserta jarak bobot sisi dalam meter. Di dalam file ini pula fungsi pencarian rute terpendek dideklarasikan lengkap dengan log pelacakan evaluasi simpul.

4.3 Logika Routing (routing.js & routing-detour.js)

Modul `routing.js` bertugas mengambil input dari UI, memanggil fungsi algoritma Dijkstra, menyusun string koordinat titik-titik lintasan, memanggil OSRM API, dan memicu penggambaran rute pada peta. Sementara itu, `routing-detour.js` menangani pemblokiran rute aktif ketika pengguna menekan tombol 'Blok Rute Ini' (misal karena macet atau jalan ditutup) untuk menghasilkan jalur alternatif cadangan.

5. PENGUJIAN SISTEM DAN VALIDASI

5.1 Metode Pengujian

Pengujian dilakukan dengan menggunakan metode Black Box Testing untuk menguji fungsionalitas antarmuka serta memvalidasi keakuratan hasil perhitungan algoritma Dijkstra dengan perhitungan matematis manual.

5.2 Hasil Pengujian Kasus

Kasus Uji	Input	Awal	&	Hasil	yang	Status
	Tujuan			Diharapkan		
Pencarian Rute Normal	Gerbang Utama ke Dekanat Teknik			Menghasilkan rute terpendek yang logis, visualisasi jalur aktif, dan rincian navigasi.		SUKSES (PASS)
Asal & Tujuan Sama	Gedung Rektorat ke Gedung Rektorat			Sistem mendeteksi kesamaan input dan menampilkan pesan error 'Titik awal dan tujuan tidak boleh sama.'		SUKSES (PASS)
Lokasi GPS (Luar UNIB)	Lokasi Saya ke Dekanat MIPA			Sistem memeriksa jarak koordinat GPS. Jika di luar UNIB, menampilkan error jangkauan.		SUKSES (PASS)

6. KESIMPULAN DAN REKOMENDASI

6.1 Kesimpulan

Berdasarkan hasil analisis, perancangan, dan pengujian sistem, dapat disimpulkan bahwa:

1. Algoritma Dijkstra berhasil diterapkan dengan akurasi 100% dalam mencari lintasan terpendek pada graf pemodelan kampus Universitas Bengkulu.
2. Integrasi dua lapis (client-side Dijkstra untuk penentuan simpul dan server-side OSRM untuk geometri jalan riil) terbukti mampu menghasilkan visualisasi navigasi yang realistis di atas peta Leaflet.js.
3. Fitur penunjang seperti pemilihan moda transportasi, pelacakan GPS, simulasi pemblokiran jalan, dan visualisasi log perhitungan memberikan nilai tambah akademis yang besar.

6.2 Rekomendasi Pengembangan

Untuk pengembangan aplikasi KibokGO UNIB ke depan, disarankan beberapa peningkatan berikut:

1. Menambahkan node persimpangan jalan kecil di dalam kampus agar cakupan rute menjadi lebih rapat dan detail.
2. Mengintegrasikan penyimpanan database di sisi backend untuk mengelola autentikasi pengguna secara aman serta pencatatan riwayat rute favorit.
3. Menyediakan fitur suara panduan arah (voice guidance) untuk memudahkan navigasi langsung pengguna di lapangan.
4. Menerapkan algoritma pencarian jalur heuristik seperti A* (A-Star) sebagai bahan komparasi performa dan waktu komputasi dengan Dijkstra.